Question 1: Lock Compatibility Matrix

Lock Compatibility Matrix:

	R	S	D	W
R	Y	N	N	N
S	N	N	N	N
D	N	N	Y	Y
W	N	N	Y	Y

Explanation:

- Operations R (read) and S (set/write) behave like standard read and write locks.
- R is only compatible with R since multiple reads do not conflict.
- S conflicts with all operations since it overwrites the value.
- D (deposit) and W (withdraw) both modify the balance, hence they conflict with R and S.
- However, D and W are **commutative operations**, meaning:

$$D(x) + W(y) = W(y) + D(x)$$

- Therefore, deposits and withdrawals do not conflict with each other or themselves.
- Hence, D and W are mutually compatible.

Note: We do not consider negative balance constraints.

Question 2: Conflict Serializability

Given Schedule:

$$R_2(d)\ R_1(a)\ W_1(a)\ R_3(b)\ R_3(c)\ R_2(a)\ W_2(a)\ R_2(c)\ W_3(c)\ R_1(d)\ R_3(d)\ W_1(d)$$

Conflicting Operations and Edges:

- **On data item a:**
  - $W_1(a)$ before $R_2(a) \Rightarrow$ edge $T_1 \rightarrow T_2$

- $W_1(a)$ before $W_2(a) \Rightarrow$ edge $T_1 \to T_2$

- **On data item c:**
  - $R_2(c)$ before $W_3(c) \Rightarrow$ edge $T_2 \to T_3$

- **On data item d:**
  - $R_2(d)$ before $W_1(d) \Rightarrow$ edge $T_2 \to T_1$
  - $R_3(d)$ before $W_1(d) \Rightarrow$ edge $T_3 \to T_1$

Precedence Graph:
Vertices: T_1, T_2, T_3
Edges:

$$T_1 \to T_2, \quad T_2 \to T_3, \quad T_2 \to T_1, \quad T_3 \to T_1$$

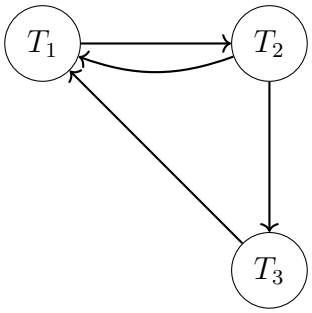
Cycle Detection:
We observe:

$$T_1 \to T_2 \to T_1$$

Hence, there is a **cycle**.
Conclusion:

- The schedule is **NOT conflict-serializable**.
- Since 2PL only produces conflict-serializable schedules,
- **2PL cannot generate this schedule**.

Precedence Graph:



Question 3: Need for Recovery Manager

Answer:

Even if system failures are guaranteed not to occur, transaction aborts can still happen due to:

- Concurrency control mechanisms (e.g., deadlock resolution in 2PL)
- User-initiated aborts (e.g., interrupt/Control-C)

Thus, a recovery manager is still required.

Case 1: Immediate Update

- Updates are written to disk before commit.
- If a transaction aborts, its effects may already be on disk.
- Therefore, **UNDO is required**.

Case 2: Deferred Update

- Updates are written to disk only after commit.
- Aborted transactions never reach disk.
- Hence, **no recovery action is required**.

Conclusion:

- Recovery manager is necessary in immediate update systems.
- It can be omitted in deferred update systems under the given assumption.

Question 4: Recoverability Analysis

Answer:

The system follows the **Write-Ahead Logging (WAL)** protocol:

- Log records are flushed before corresponding data is written to disk.
- At commit, all updates are forced to disk before writing commit record.

Recoverability:

Yes, the system is recoverable because WAL ensures consistency between log and disk.

Recovery Actions:

- **UNDO Required:**
  - Since updates are written before commit (immediate update),
  - Aborted transactions may have modified disk state,
  - Hence, their effects must be undone.
- **REDO Not Required:**

- At commit, all updates are forced to disk,
- Therefore, committed transactions are already reflected on disk.

Conclusion:

UNDO AND NO-REDO protocol